14.5 Infraestrutura de produção — status de entrega

Bloco 14.5 Infraestrutura de produção · Sprint 14 — Migração da planilha e go-live · Data: 2026-10-26 · Branch: `main` · Ambiente: gc-dev (`gzbmhgnpoehormnidmgg`), como ensaio

Resumo

Ensaio executado em gc-dev em 2026-10-26, com os números reais abaixo. O Breno não tem acesso ao gc-prod e decidiu em 26/10: "não vamos fazer nada em gc-prod, finalize tudo pelo dev". O que dá para provar em gc-dev (backup, teste de restauração, monitoramento existente) foi executado; o que só existe em prod (domínio, Vercel de prod, backup do plano pago, Sentry) fica como registro. gc-prod fora do escopo por decisão do Breno (26/10): o bloco fecha em gc-dev; o restante é registro para quem for operar prod. A documentação da restauração já existia: é a seção 12 de `docs/tecnicos/runbook-producao.md`, agora exercitada.

Estado conferido em 2026-10-26 pela API de gerenciamento da Supabase: gc-prod `INACTIVE`, gc-dev `ACTIVE_HEALTHY`, ambos Postgres 17.6.1.105. O repositório não tem `vercel.json`, não tem `@sentry/*` no `package.json` e não tem configuração de backup.

1. As atividades do card

ATIVIDADE	ESTADO	EVIDÊNCIA
Backup automático ativo e teste de restauração	**teste de restauração executado em gc-dev.** O backup automático do plano do gc-prod é só registro	seção 1.1
Domínio próprio na Vercel com HTTPS	**só registro:** depende do domínio e do DNS de prod	runbook, seção 11
Monitoramento de erro (Sentry ou log da Vercel)	**registro do que existe em gc-dev**; Sentry fica como registro (decisão do 14.1)	seção 1.2
Documentar a restauração	✅ documentado e **agora testado**, com o tempo medido	runbook, seção 12, e o rollback na seção 14

1.1 Teste de restauração (gc-dev, 2026-10-26)

PASSO	RESULTADO
Dump do schema, `npx supabase db dump --linked` (21:55)	163.656 bytes, 1 min 37 s (inclui o pull da imagem do `pg_dump`)
Dump dos dados, `--data-only --use-copy` (21:57)	3.071.438 bytes, 17,7 s; traz `public`, `auth` e `storage`
Destino	container Docker descartável `public.ecr.aws/supabase/postgres:17.6.1.105`, a mesma versão do gc-dev, porta 54999
Restauração com `psql`: `schema.sql`, depois `dados.sql` (21:58)	**schema 1 s + dados menos de 1 s: cerca de 2 s no total**, medido por relógio de parede em segundos inteiros. Schema: 0 erros
Contagem das 20 tabelas de `public`, gc-dev × restaurado	**20/20 iguais:** acordo_parcelas 10, acordos_pagamento 5, auditoria_eventos 2575, automacao_eventos 87, clientes 57, contatos_whatsapp 3, contratos 10, documentos_processamento 57, empresas 2, execucao 53, execucao_medicoes 19, fd 5, itens 76, notas_fiscais 8, obras 4, orcamentos 28, pagamentos 9, pedidos_acesso 0, profiles 8, propostas 17
Container derrubado	`docker rm -f gc-restore-teste`; nenhum container sobrando

O que o teste **achou** (vale para quem operar prod):

1. **Só `public` restaura limpo.** O dump de dados leva `auth` e `storage`, mas o schema deles é do Supabase, e o dump de schema não o recria. Os `COPY` de `auth` e `storage` falharam no destino (33 erros no log, todos desses dois schemas: colunas e tabelas de versão diferente, e tabelas inexistentes). `auth.users` ficou com 0 linhas no destino, contra 8 em gc-dev. Para recuperar usuários num incidente, o caminho é o *Restore* do painel ou o PITR (o mesmo projeto), ou recriar os usuários por convite; o dump lógico cobre as tabelas do sistema.
2. **As FKs de `profiles` para `auth.users` não bloquearam** porque o dump de dados liga `session_replication_role = replica`. Quem restaurar sem isso precisa da ordem `auth` antes.
3. **O Storage (anexos) não está em nenhum dump** e não foi baixado neste ensaio.
4. **Tempo de sistema parado:** a restauração de `public` leva segundos nesse volume (3 MB). O que pesa num incidente é o tempo de reativar o projeto e subir o Storage, não o `psql`. Em prod o volume deve ser maior, e o tempo deve ser medido de novo.

Os arquivos ficaram em `~/.claude/jobs/2a37dcc8/tmp/backup-gcdev/` (fora do repositório).

1.2 Monitoramento que já existe em gc-dev (consulta de 2026-10-26)

FONTE	O QUE MOSTRA	NÚMEROS REAIS
`auditoria_eventos` (13.2)	toda escrita, com `resultado`	2.635 eventos: 2.634 `sucesso` e **1** **`erro`** (2026-09-23). **Últimos 7 dias: 63 eventos, todos `sucesso` (60 da automação, 3 do banco), 0 erros**
`automacao_eventos` (13.2, tela `/logs`)	n8n e bot, por etapa e nível	87 eventos: 73 info, 12 aviso e **2 erro** (o último em 2026-10-01). **Últimos 7 dias: 0 eventos**
Logs do Supabase (painel)	API, Postgres, Auth	não consultados nesta rodada (painel; sem API usada aqui)
Sentry	erros da aplicação no navegador e no servidor	**registro:** não instalado; decisão do 14.1
Log da Vercel	erros do servidor	**registro:** depende do projeto de prod

As duas tabelas cobrem erro de escrita e da automação, não erro de renderização da aplicação nem erro no navegador; isso é o que o Sentry ou o log da Vercel resolveriam.

2. O que já se sabe

- **Dois deploys na Vercel, segundo os documentos.** O `gc-sistema-nine.vercel.app`, do fork `brenocmlo/gc-sistema`, aponta para o gc-dev. O `.github/workflows/sincronizar-fork.yml` o mantém igual à `main` do principal. O outro, do repositório principal (`cremalet`), é citado como ligado ao gc-prod e protegido por login da Vercel. Nenhum dos dois foi aberto nesta entrega.
- **O convite por link não depende do domínio no Auth.** O link é montado com o endereço por onde o admin acessou e validado com `verifyOtp` (`src/app/convite/route.ts`). Com o domínio próprio, os links saem nele sem configuração extra.
- **O log da automação já existe** (`automacao_eventos`, tela `/logs`, 13.2). Ele cobre o n8n e o bot, mas não os erros da aplicação. É isso que o Sentry ou o log da Vercel resolveriam.
- **O CI do 13.5** (`.github/workflows/validar.yml`) ainda não está no GitHub (registro do 13.5). Ele não é monitoramento de produção, mas é o outro aviso de erro que falta ligar.

3. O que a equipe precisa decidir ou fornecer

1. **O plano da Supabase do gc-prod tem backup automático diário?** Se não tiver, contratar? E o PITR (restaurar para um horário exato), que é um adicional pago?
2. **O domínio** e quem tem acesso ao DNS.
3. **Sentry ou log da Vercel** para os erros da aplicação, e quem recebe o aviso.
4. **Qual projeto da Vercel vira produção** (o mesmo item do 14.4).

5. ~~Autorização do Breno para gc-prod~~ — gc-prod fora do escopo por decisão do Breno (26/10): o bloco fecha em gc-dev; o restante é registro para quem for operar prod. Falta só um projeto descartável na Supabase para o teste de restauração, a partir de um dump do gc-dev.

4. Validação

Não rodou código novo: o bloco só executou procedimentos e consultas. Números reais:

COMANDO	SAÍDA
`GET https://api.supabase.com/v1/projects`	gc-prod `INACTIVE`, gc-dev `ACTIVE_HEALTHY`, os dois 17.6.1.105
`npx supabase db dump --linked` (schema, dados)	163.656 e 3.071.438 bytes
restauração em container `supabase/postgres:17.6.1.105`	20/20 tabelas de `public` iguais; ~2 s; container removido
consultas de `auditoria_eventos` e `automacao_eventos`	0 erros nos últimos 7 dias nas duas
`ls vercel.json`; `grep sentry package.json`	não existe; nada
`bash scripts/validacao/validar.sh build fumaca` (a fumaça é a do 14.4)	22/22 passos ok

A rodada completa de 2026-10-27 (`npm run validar`) passou nas sete camadas: unitário 432, build 66 rotas, runtime 327/327, dados 65/65, escrita 529/529, navegador 281/281.

5. Pendências

- **Backup automático do plano do gc-prod:** registro (item 1 da seção 3). Em gc-dev não foi verificado no painel se o plano tem backup diário ou PITR.
- **Teste de restauração em prod:** registro. Em gc-dev ficou provado para `public`; `auth` e `storage` não restauram por dump lógico (seção 1.1), e o Storage não foi baixado.
- **Domínio e HTTPS:** registro; fora do escopo (prod).
- **Monitoramento:** Sentry ou log da Vercel não instalados nem escolhidos; registro.
- **Logs do Supabase no painel** não consultados.
- **Índice:** a linha do 14.5 está em `docs/README.md`.